FACULTAD DE INGENIERÍA

Escuela Académico Profesional de Ingeniería de Sistemas e Informática

Proyecto de Investigación

Evaluación del Efecto de un Framework de Ingeniería de Prompts Basado en la Norma ISO/IEC 25010 en la Mantenibilidad y la Deuda Técnica del Código Generado por Inteligencia Artificial Generativa

AUTORES:

Harold Huanca Ccasa

Jose Leonardo Cruz Gonzales

Wilder Ronald Ccasani Quispe

Cusco, Perú

2026

ÍNDICE

INTRODUCCIÓN **v**

1. EL PROBLEMA DE INVESTIGACIÓN **1**
1.1. Planteamiento del Problema 1
1.2. Formulación del Problema 3
1.2.1. Problema General 3
1.2.2. Problemas Específicos 3
1.3. Objetivos de la Investigación 3
1.3.1. Objetivo General 3
1.3.2. Objetivos Específicos 3
1.4. Justificación de la Investigación 3
1.4.1. Justificación Teórica 3
1.4.2. Justificación Práctica 4
1.4.3. Justificación Metodológica 4
1.5. Delimitación de la Investigación 4
1.5.1. Delimitación Espacial 4
1.5.2. Delimitación Temporal 4
1.5.3. Delimitación Social 5
1.6. Hipótesis y Variables 5
1.6.1. Hipótesis 5
1.6.2. Definición de Variables 5
1.6.3. Operacionalización de Variables 6

2. MARCO TEÓRICO **7**
2.1. Antecedentes de la investigación 7
2.1.1. Antecedentes Internacionales 7
2.1.2. Antecedentes Nacionales 9
2.2. Bases teóricas 12
2.2.1. Marco Conceptual 12
2.2.1.1. Ingeniería de Prompts (Prompt Engineering) 12
2.2.1.2. Mantenibilidad del Software 12
2.2.1.3. Deuda Técnica (Technical Debt) 12
2.2.1.4. Complejidad Ciclomática de McCabe 12
2.2.1.5. PromptDebt 12
2.2.1.6. Análisis Estático de Código 12
2.2.1.7. Deuda Técnica Autoadmitida (SATD) 13
2.2.2. Bases Teóricas 13
2.2.2.1. Ingeniería de Software y Calidad del Producto 13
2.2.2.2. Inteligencia Artificial Generativa en el Desarrollo de Software 13
2.2.2.3. Paradigma de la Ingeniería de Prompts (Prompt Engineering) 14
2.2.2.4. Deuda Técnica en Entornos Generativos 14

2.2.2.5. Marcos de Trabajo y Evaluación Automatizada 15

3. METODOLOGÍA 16

3.1. Método, tipo y alcance de la investigación 16

3.1.1. Método de investigación 16

3.1.2. Tipo de investigación 16

3.1.3. Alcance de la investigación 16

3.1.4. Diseño de la investigación 16

3.1.5. Enfoque de la investigación 16

3.2. Materiales y métodos 17

3.2.1. Materiales 17

3.2.1.1. Estación de Trabajo de Alto Rendimiento 17

3.2.1.2. Entorno de Ejecución y Nodos Computacionales 17

3.2.2. Recursos Lógicos y Modelos de Inteligencia Artificial 17

3.2.2.1. Modelos de Lenguaje de Gran Escala (LLMs) de Acceso Abierto 17

3.2.2.2. Entorno de Orquestación y Contenerización 18

3.2.3. Herramientas de Análisis Estático y Desarrollo 18

3.2.3.1. Framework de Evaluación de Calidad de Código 18

3.2.3.2. Ecosistema de Lenguajes de Programación Objetivo 19

3.2.4. Métodos 19

3.2.4.1. Conceptualización del Framework basado en ISO/IEC 25010 19

3.2.4.2. Aplicación de Patrones Arquitectónicos de Prompts 19

3.2.5. Desarrollo y Generación de Código de Microservicios 19

3.2.5.1. Configuración del Entorno de Generación 19

3.2.5.2. Ejecución de Pruebas de Generación 19

3.2.6. Pipeline de Análisis Estático y Validación de Calidad 20

3.2.6.1. Proceso de Evaluación de Métricas de Código 20

3.2.6.2. Validación y Control de Calidad 20

3.3. Sustento Teórico de la Metodología 20

4. ASPECTOS ADMISNITRATIVOS 21

4.1. Presupuesto 21

4.2. Cronograma 21

5. RESULTADOS Y DISCUSIÓN 23

5.1. Presentación de resultado 23

5.2. Discusión de resultados 23

6. CONCLUSIONES Y RECOMENDACIONES 24

ANEXOS 25

Anexo 1. Matriz de Consistencia 25

Referencias Bibliográficas 26

ÍNDICE DE TABLAS

1. Operacionalización de Variables 6
2. Especificaciones técnicas de la estación de trabajo. 17
3. Modelos de lenguaje seleccionados para la investigación. 18
4. Estructura del presupuesto ajustado del proyecto. 21

ÍNDICE DE FIGURAS

1. Cronograma del proyecto elaborado en ProjectLibre 22

INTRODUCCIÓN

CAPÍTULO 1
EL PROBLEMA DE INVESTIGACIÓN

1.1 Planteamiento del Problema

A nivel global, la integración de la Inteligencia Artificial Generativa (IAG) en el ciclo de vida del desarrollo de software ha transformado la productividad de la industria; no obstante, ha introducido un fenómeno crítico denominado "deuda técnica automatizada". Investigaciones recientes como la de Hou et al. [1] sostienen que el código generado por modelos de lenguaje de gran escala (LLMs) a menudo carece de una visión arquitectónica integral, derivando en sistemas con una complejidad estructural elevada. Esta tendencia sugiere un incremento sustancial en la velocidad de codificación, pero a costa de una degradación en la mantenibilidad, lo que contraviene los estándares de calidad interna establecidos por la norma ISO/IEC 25010 [2]. Diversos estudios han evidenciado que el código generado por modelos de lenguaje de gran escala presenta limitaciones en términos de mantenibilidad y calidad estructural. Por ejemplo, investigaciones recientes señalan que el uso de LLMs en generación de código puede introducir complejidad innecesaria y deuda técnica acumulada, especialmente cuando no se aplican lineamientos estructurados de diseño o validación [3, 4]. Estos hallazgos evidencian la necesidad de establecer marcos metodológicos que orienten la generación de código hacia estándares de calidad definidos.

En el contexto nacional, el Perú atraviesa un proceso de aceleración digital orientado a la optimización de procesos mediante la automatización. Según el [5], existe un incremento sustancial en la adopción de servicios tecnológicos en el sector empresarial; sin embargo, este avance presenta desafíos estructurales. De acuerdo con la [6], el país enfrenta una brecha en la formación de talento especializado y en la definición de estándares de gobernanza técnica que aseguren la calidad de las soluciones de IA. Esta carencia de marcos de validación estandarizados eleva el riesgo de acumular deuda técnica en sistemas críticos, comprometiendo la mantenibilidad y la seguridad del software desarrollado en el entorno nacional.

A nivel local, se observa en el entorno académico y profesional de la región Cusco una adopción empírica de la Inteligencia Artificial Generativa para la resolución de problemas de programación. A través de la experiencia en el desarrollo de proyectos de software dentro de la Universidad Continental, se ha identificado que la prioridad suele centrarse en la funcionalidad inmediata, postergando la calidad interna del código. Esta observación coincide con las brechas identificadas en el [7], donde se evidencia que la transformación digital de la región requiere de una mayor rigurosidad en los estándares técnicos. Por lo tanto, la presente investigación surge de la necesidad de establecer herramientas que transformen esta práctica empírica observada en la formación profesional en un proceso estandarizado que asegure la mantenibilidad según la norma ISO/IEC 25010.

El problema central se identifica como la insuficiente calidad estructural y la elevada deuda técnica presente en el software generado mediante Inteligencia Artificial Generativa (IAG) ante la ausencia de marcos metodológicos de ingeniería. Investigaciones previas [3] demuestran que, aunque los modelos de lenguaje producen código funcional, este

suele presentar deficiencias en los atributos de mantenibilidad y complejidad ciclomática definidos por la norma ISO/IEC 25010. Asimismo, la carencia de un método sistemático para estructurar los requerimientos técnicos o prompts deriva en lo que la literatura denomina como opacidad técnica o "cajas negras" de código [8], dificultando los procesos de auditoría, optimización y escalabilidad del software en entornos de producción.

Las causas de esta problemática son multidimensionales. En el plano técnico, se observa una dependencia de modelos de IA que, a pesar de su capacidad funcional, generan un incremento del 34 % en la complejidad ciclomática y una redundancia lógica 2.1 veces superior a la del código humano [9], evidenciando que los LLMs requieren de una guía estructural y arquitectónica para asegurar la mantenibilidad [10]. A nivel organizacional, la presión por reducir los tiempos de entrega (*Time-to-Market*) fomenta la acumulación de deuda técnica autoadmitida (*Self-Admitted Technical Debt* - SATD) y la omisión de fases críticas de validación [11, 12]. Finalmente, en el factor humano, persiste una carencia de competencias en ingeniería de prompts aplicada sistemáticamente, lo que deriva en una degradación de la calidad denominada *PromptDebt* [4], limitando el uso de la IA a una herramienta de consulta en lugar de integrarla como un componente robusto en un *pipeline* de producción profesional [13, 14].

Las consecuencias de persistir en este modelo empírico de desarrollo son graves. Operativamente, las organizaciones enfrentarán un incremento exponencial en los costos de mantenimiento preventivo y correctivo, debido a que el código generado por IA sin supervisión técnica demanda un esfuerzo de refinamiento masivo para alcanzar estándares profesionales [15]. Económicamente, la deuda técnica acumulada puede superar el valor del activo de software, pues el incremento en la complejidad y la redundancia lógica elevan los costos de evolución del sistema hasta volverlo insostenible [9, 16]. Socialmente, la baja confiabilidad de los sistemas pone en riesgo la integridad de datos sensibles, comprometiendo la confianza en la digitalización nacional y regional [6, 7]. Como advierte la literatura reciente, no se trata de errores sintácticos aislados, sino de una degradación estructural o *PromptDebt* que compromete directamente la resiliencia y la competitividad organizacional [4, 11].

De no establecerse un **Framework de Ingeniería de Prompts** que actúe como un filtro de calidad basado en la norma [2], la industria del software se enfrentará a una saturación de sistemas heredados (*legacy*) generados por IA que resultarán inmanejables a mediano plazo [1, 4]. Prospectivamente, el éxito de la ingeniería de sistemas en la próxima década dependerá de la capacidad de los profesionales para sistematizar la interacción con los LLMs, transformando la generación de código de un proceso aleatorio a una disciplina de ingeniería predictiva y controlada [13, 14]. Solo mediante la adopción de estrategias de gobernanza y métricas de calidad adaptadas a la era generativa se podrá asegurar que la tecnología cumpla con los objetivos de transformación digital nacional, actuando como un motor de eficiencia y no como un generador de inestabilidad sistémica en infraestructuras críticas [6, 17].

La unidad de análisis está constituida por módulos de código fuente generados por modelos de lenguaje de gran escala (LLMs), específicamente implementaciones de microservicios

desarrolladas en lenguajes de programación como Python y TypeScript. Cada unidad será evaluada mediante métricas de calidad interna para determinar su nivel de mantenibilidad y deuda técnica.

1.2 Formulación del Problema

1.2.1 Problema General

¿Qué efecto tiene la implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 en la mantenibilidad y la deuda técnica del código generado por inteligencia artificial generativa?

1.2.2 Problemas Específicos

PE1 ¿Qué efecto tiene la aplicación de patrones arquitectónicos de prompts en la complejidad ciclomática de McCabe del código generado?

PE2 ¿Qué efecto tiene la implementación de un pipeline de análisis estático en la reducción de la deuda técnica del software generado?

PE3 ¿Qué efecto tiene la aplicación del protocolo de validación basado en ISO/IEC 25010 en el índice de mantenibilidad del código generado?

1.3 Objetivos de la Investigación

1.3.1 Objetivo General

Evaluar el efecto de la implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 en la mantenibilidad y la deuda técnica del código generado por inteligencia artificial generativa.

1.3.2 Objetivos Específicos

OE1 Evaluar el efecto de la aplicación de patrones arquitectónicos de prompts en la complejidad ciclomática de McCabe.

OE2 Evaluar el efecto de la implementación de un pipeline de análisis estático en la reducción de la deuda técnica.

OE3 Evaluar el efecto de la aplicación del protocolo de validación basado en ISO/IEC 25010 en el índice de mantenibilidad.

1.4 Justificación de la Investigación

1.4.1 Justificación Teórica

La investigación se justifica teóricamente al proponer una convergencia necesaria entre la Ingeniería de Software clásica y el paradigma emergente de la Inteligencia Artificial Generativa (IAG). Ante el fenómeno de la çaja negra"que representan los LLMs, este estudio aporta un marco conceptual que vincula el determinismo de la norma ISO/IEC 25010 con la naturaleza estocástica del lenguaje natural. Al integrar la Teoría de la Complejidad de McCabe con la estructuración de instrucciones, se genera un nuevo conocimiento sobre cómo la semántica del *prompt* impacta directamente en el grafo de flujo de control del software resultante. Este trabajo

expande la frontera del conocimiento al tratar a la IA no como un ente aislado, sino como un agente dentro de un sistema sociotécnico que requiere de gobernanza arquitectónica para mitigar la entropía del código.

1.4.2 Justificación Práctica

En la práctica, el estudio aborda la crisis de mantenibilidad generada por el uso empírico de la IAG en la industria. Al proporcionar un Framework validado, se ofrece una solución tangible al problema de la "PromptDebt", permitiendo que las organizaciones reduzcan el tiempo de refactorización manual y los costos operativos de soporte de nivel 3 [16]. La implementación de este marco transforma el uso de la IA de un asistente de autocompletado a un motor de generación de microservicios profesionalmente aptos para infraestructuras críticas, garantizando que el ahorro inicial en tiempo de codificación no se traduzca en una deuda técnica impagable a mediano plazo [9].

1.4.3 Justificación Metodológica

Metodológicamente, la investigación establece un precedente de auditoría técnica para el código sintético. Se justifica mediante el diseño de un *pipeline* de evaluación experimental que elimina la subjetividad en la apreciación de la calidad del código. Al combinar herramientas de análisis estático (como SonarQube) con protocolos de validación normativa, se propone un modelo de experimentación replicable y escalable. Este procedimiento permite transformar la Ïngeniería de Prompts"de una técnica de ensayo y error en una disciplina de ingeniería medible, auditable y controlada, proporcionando métricas precisas para la toma de decisiones en la gestión de proyectos tecnológicos.

1.5 Delimitación de la Investigación

1.5.1 Delimitación Espacial

El estudio se desarrollará en el Laboratorio de Ingeniería de Sistemas de la Universidad Continental, sede Cusco. La infraestructura tecnológica consistirá en estaciones de trabajo bajo el sistema operativo Arch Linux, configuradas para la gestión de contenedores y despliegue de microservicios. Se utilizarán modelos de lenguaje de última generación a través de APIs y entornos locales para garantizar la trazabilidad de los datos y minimizar la latencia de red, asegurando un entorno de pruebas controlado y aislado de interferencias externas.

1.5.2 Delimitación Temporal

La investigación es de corte transversal y se ejecutará durante el periodo académico 2026-I, abarcando desde marzo hasta julio de 2026. Este periodo permite una observación secuencial que inicia con la delimitación algorítmica [13], continúa con la fase de experimentación intensiva y concluye con el análisis estadístico de la deuda técnica acumulada frente a los umbrales de la norma ISO 25010.

1.5.3 Delimitación Social

El alcance social del estudio se circunscribe a los actores del ecosistema de desarrollo de software: ingenieros de sistemas, arquitectos de soluciones y especialistas en aseguramiento de la calidad (QA). Se enfoca particularmente en la comunidad técnica de la región Cusco, buscando profesionalizar el uso de la IAG en las Mypes tecnológicas y en la formación de futuros ingenieros, alineándose con las estrategias nacionales de transformación digital y ética en IA [6, 18].

1.6 Hipótesis y Variables

1.6.1 Hipótesis

Hipótesis General: La implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 reduce significativamente la deuda técnica y mejora la mantenibilidad del código generado por inteligencia artificial generativa en comparación con métodos empíricos.

Hipótesis Específicas:

HE1 El código generado mediante el uso de patrones arquitectónicos de prompts presenta menor complejidad ciclomática que el código generado mediante métodos empíricos.

HE2 El código generado con un pipeline de análisis estático presenta menor deuda técnica que el generado sin dicho pipeline.

HE3 El código generado bajo el protocolo de validación basado en ISO/IEC 25010 presenta un mayor índice de mantenibilidad que el código generado sin dicho protocolo.

1.6.2 Definición de Variables

Variable Independiente (X): Aplicación del Framework de Ingeniería de Prompts

Variables Dependientes (Y):

VD1 **Mantenibilidad del Software:** Medida a través del índice de mantenibilidad y la complejidad ciclomática de McCabe utilizando herramientas de análisis estático (SonarQube/Radon).

VD2 **Deuda Técnica:** Medida a través del tiempo estimado de refactorización y la densidad de defectos detectados por el *pipeline* automatizado.

1.6.3 Operacionalización de Variables

Tabla 1. Operacionalización de Variables

Variable	Dimensión	Indicador	Técnica	Instrumento
Framework de Prompts (VI)	Aplicación estructurada	Uso de patrones como Chain-of-Thought y Few-shot prompting	Experimental	Plantilla de prompts
Mantenibilidad (VD1)	Calidad interna	Índice de mantenibilidad	Análisis estático	SonarQube
	Complejidad	Complejidad ciclomática	Análisis estático	Radon
Deuda técnica (VD2)	Esfuerzo	Tiempo de remediación	Análisis automático	SonarQube
	Calidad	Densidad de defectos	Análisis automático	SonarQube

CAPÍTULO 2
MARCO TEÓRICO

2.1 Antecedentes de la investigación

2.1.1 Antecedentes Internacionales

Asfandyar, Akhtar y Khalil [9], en su investigación de 2025 titulada *Evaluating the Impact of AI Code Generated of Software Quality and Observed Study of Maintainability, Security and Performance*, examinaron el efecto de la IA generativa en la integridad técnica del software. El objetivo fue evaluar sistemáticamente la calidad del código producido por modelos como GPT-4 y Claude 3 en términos de mantenibilidad, seguridad y rendimiento. Bajo una metodología experimental cuantitativa, compararon 642 soluciones sintéticas frente a 107 humanas mediante herramientas de análisis estático como SonarQube y Bandit. Los resultados demostraron que el código generado por IA incrementó la complejidad ciclomática en un 34

Aljohani y Do [4], en su investigación de 2025 titulada *PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects*, analizaron el impacto del código técnico y los artefactos de IA en la mantenibilidad de sistemas modernos. El estudio tuvo como objetivo realizar el primer análisis empírico a gran escala sobre la deuda técnica autoadmitida (SATD) específica para modelos de lenguaje. Mediante el examen de 93,142 archivos de Python, los autores identificaron que el 54.49

Azeem, Naveed, Sajid y Ali [15], en su estudio de 2025 titulado *AI vs. Human Programmers: Complexity and Performance in Code Generation*, evaluaron si la inteligencia artificial ha avanzado lo suficiente como para reemplazar a los programadores humanos. La investigación comparó 300 programas de Python generados por ChatGPT (GPT-4) con códigos funcionalmente equivalentes escritos por expertos humanos, utilizando métricas cuantitativas como la Complejidad de Halstead y la Complejidad Ciclomática de McCabe, además de una evaluación cualitativa por pares. Los resultados cuantitativos revelaron que el código de la IA es significativamente más verboso y complejo, con un aumento del 20.8

Cordeiro, Noei y Zou [19], en su estudio de 2024 titulado *An Empirical Study on the Code Refactoring Capability of Large Language Models*, analizaron la capacidad de los modelos de lenguaje para realizar refactorización automática de código y mejorar su calidad interna. El objetivo principal fue evaluar si los LLMs pueden optimizar métricas de calidad, reducir code smells y mantener la funcionalidad del software. Se empleó una metodología cuantitativa de diseño experimental, analizando 30 proyectos open source en Java con más de 5,000 commits de refactorización, utilizando modelos como GPT-4o, LLaMA 3 y DeepSeek-v3. La recolección de datos se realizó mediante la aplicación de técnicas de prompting como zero-shot, one-shot y chain-of-thought, evaluando métricas como complejidad, cohesión y acoplamiento, además de pruebas unitarias para validar la funcionalidad. Los resultados evidenciaron que los LLMs lograron reducir significativamente los code

smells y mejorar la calidad estructural del código, destacando mejoras en cohesión, acoplamiento y complejidad ciclomática. Asimismo, se identificó que técnicas de prompt engineering, especialmente chain-of-thought, incrementaron la efectividad de la refactorización y la tasa de éxito en pruebas unitarias. Se concluyó que, si bien los modelos son efectivos en tareas de refactorización, su desempeño depende directamente de la calidad y estructura de los prompts utilizados. Este estudio resulta relevante para la investigación actual, ya que valida que la aplicación sistemática de patrones de prompts influye directamente en la calidad del código generado, sustentando la hipótesis de que un framework de ingeniería de prompts puede mejorar la mantenibilidad y reducir la deuda técnica en sistemas generados por inteligencia artificial.

Cuatecontzi Cuahutle, Medina Barrera, Cortes Maldonado y Bueno Avendaño [12], en su revisión sistemática de 2023 titulada *Identificación y Medición de Deuda Técnica Autoadmitida en Herramientas de Aprendizaje Profundo*, analizaron la presencia y características de la deuda técnica autoadmitida en sistemas basados en inteligencia artificial. El objetivo del estudio fue identificar los tipos de SATD y los métodos utilizados para su detección y medición en el contexto del aprendizaje profundo. La metodología correspondió a un enfoque cualicuantitativo basado en una revisión sistemática de la literatura, aplicando criterios de inclusión y exclusión sobre estudios primarios, así como el análisis de comentarios de código y registros de commits como principales fuentes de datos. Los resultados evidenciaron que la deuda técnica autoadmitida se presenta con mayor frecuencia en componentes relacionados con el diseño del modelo, la implementación de algoritmos y la gestión de datos, afectando principalmente atributos de calidad como la mantenibilidad, portabilidad y comprensión del código. Asimismo, se identificaron diversas técnicas para su detección, destacando el uso de análisis de lenguaje natural en comentarios y herramientas automatizadas. Se concluyó que la SATD representa un problema crítico en sistemas inteligentes, debido a su impacto acumulativo en la calidad del software. Este antecedente resulta relevante para la investigación actual, ya que proporciona un sustento teórico sobre la identificación y medición de la deuda técnica autoadmitida, variable clave en el estudio, y respalda el desarrollo de un framework basado en ingeniería de prompts orientado a su reducción y mejora de la mantenibilidad del código generado.

Siddiq y Santos [8], en su estudio de 2023 titulado *Exploring the Effectiveness of Large Language Models in Generating Unit Tests*, analizaron la capacidad de los modelos de lenguaje para generar pruebas unitarias y su impacto en la calidad del software. El objetivo fue evaluar la efectividad de estos modelos en términos de cobertura, precisión y capacidad de ejecución de las pruebas generadas. Se empleó una metodología cuantitativa de tipo experimental, utilizando fragmentos de código de distintos repositorios para generar automáticamente casos de prueba mediante modelos de lenguaje. La recolección de datos se basó en la ejecución de las pruebas generadas, evaluando su tasa de éxito, cobertura de código y detección de errores.

Los resultados evidenciaron que, si bien los modelos de lenguaje son capaces de generar pruebas unitarias funcionales, una proporción significativa de estas falla debido a la falta de contexto estructural y comprensión del comportamiento del sistema. Asimismo, se identificó que la calidad de las pruebas mejora cuando se proporcionan instrucciones más específicas mediante técnicas de prompt engineering. Se concluyó que la testabilidad, como subcaracterística de la mantenibilidad según la norma ISO 25010, representa un desafío crítico en la generación automática de código asistida por IA. Este estudio resulta relevante para la investigación actual, ya que sustenta la inclusión de mecanismos de validación basados en estándares de calidad y refuerza la hipótesis de que el uso de prompts estructurados puede mejorar la mantenibilidad del código, específicamente en la subcaracterística de capacidad de ser probado.

2.1.2 Antecedentes Nacionales

Romero Aguilar, Vargas Alarcón, Balboa Zegarra, Alvarado Sánchez y Plasencia Sifuentes [16], en su investigación de 2024 titulada *Business Consulting: Aplicaciones de marcos de referencia para el desarrollo de software en MS4M S.A.C.*, analizaron la problemática de la deuda técnica en una empresa del sector tecnológico peruano. El objetivo del estudio fue proponer lineamientos de gobernanza y buenas prácticas basadas en marcos de referencia para reducir los costos asociados al mantenimiento de software. La metodología correspondió a un estudio de caso descriptivo con enfoque cualitativo, en el cual se recolectaron datos mediante diagnósticos técnicos, revisión documental y análisis de procesos dentro de una empresa dedicada al desarrollo de soluciones para el sector minero. Los resultados evidenciaron que la ausencia de una arquitectura estandarizada y de lineamientos formales de desarrollo incrementa significativamente los tiempos de mantenimiento, generando retrabajo y acumulación de deuda técnica. Asimismo, se identificó que la implementación de marcos de referencia arquitectónicos permite mejorar la organización del código, optimizar los procesos de desarrollo y reducir los costos operativos. Se concluyó que la adopción de prácticas de gobernanza y arquitectura empresarial es fundamental para garantizar la sostenibilidad y calidad del software. Este antecedente resulta relevante para la investigación actual, ya que contextualiza la problemática de la deuda técnica en el entorno peruano y respalda la necesidad de implementar soluciones estructuradas, como el uso de microservicios y frameworks basados en ingeniería de prompts, para mejorar la mantenibilidad del software y reducir la deuda técnica.

Pardo Mesías [20], en su estudio de 2018 titulado *Mantenibilidad de productos de software según el modelo SQUARE ISO/IEC 25000*, tuvo como objetivo determinar el grado de mantenibilidad de productos de software desarrollados por egresados de ingeniería en informática en el Perú. La investigación fue de tipo aplicada, con nivel descriptivo simple y diseño descriptivo, utilizando una metodología basada en estudios de caso. La población estuvo conformada por productos de softwa-

re desarrollados por egresados, seleccionándose una muestra mediante criterio de conveniencia. Para la recolección de datos se emplearon herramientas de análisis estático y métricas definidas en la norma ISO/IEC 25000, evaluando subcaracterísticas como modularidad, reusabilidad, analizabilidad, capacidad de ser modificado y capacidad de ser probado. Los resultados evidenciaron que, en términos generales, los productos analizados presentaron un nivel alto de mantenibilidad; sin embargo, se identificaron deficiencias en subcaracterísticas clave como la modularidad y la capacidad de ser probado, lo cual sugiere debilidades en la calidad interna del software. Se concluyó que la aplicación del modelo ISO/IEC 25000 permite evaluar de manera estructurada la calidad del producto, proporcionando indicadores útiles para la mejora continua del desarrollo de software. Este antecedente se relaciona con la presente investigación al sustentar el uso de modelos normativos como ISO/IEC 25010 para evaluar la mantenibilidad como variable dependiente. Además, refuerza la necesidad de incorporar enfoques estructurados, como un framework de ingeniería de prompts, para mejorar la calidad del código generado por inteligencia artificial, reduciendo la deuda técnica y optimizando las métricas de mantenibilidad.

Díaz Sifuentes, Paz Fernández y Torres Villanueva [14], en su artículo de 2024 titulado *Optimización de Modelos de Lenguaje Grande (LLMs) a través del Prompt Engineering*, tuvieron como objetivo analizar el impacto del diseño de instrucciones en la optimización del rendimiento de modelos de lenguaje como GPT y BERT. La investigación empleó un enfoque metodológico integral de carácter experimental y analítico, combinando revisión sistemática de literatura, análisis de casos prácticos, experimentos controlados y entrevistas a expertos. La población estuvo constituida por modelos de lenguaje preentrenados, evaluados mediante tareas de generación de texto, clasificación temática y resolución de problemas, utilizando métricas como precisión y coherencia en las respuestas. Los resultados evidenciaron que el diseño estratégico de prompts, especialmente mediante técnicas como *few-shot learning* y razonamiento en cadena, permitió mejorar significativamente la precisión, relevancia y adaptabilidad de las respuestas, además de reducir errores y falsos positivos. Asimismo, se observó que el prompt engineering facilita la implementación práctica de los modelos sin necesidad de ajustes internos complejos, optimizando su desempeño en distintos contextos de aplicación. Se concluyó que el prompt engineering constituye una técnica fundamental para maximizar el potencial de los LLMs, actuando como una capa de control externo que mejora su rendimiento. Este antecedente se vincula directamente con la presente investigación, ya que sustenta la variable independiente al demostrar que la estructuración de prompts permite influir en la calidad del código generado, contribuyendo a mejorar la mantenibilidad y reducir la deuda técnica.

Medianero Acosta [13], en su trabajo de 2024 titulado *Algoritmos imperativos como insumo para generar código de programación aplicando técnicas de Ingeniería Rápida en Inteligencia Artificial*, tuvo como objetivo analizar el uso de algoritmos imperativos como base estructurada para la generación automatizada de código

mediante modelos de lenguaje. La investigación fue de tipo aplicada con enfoque técnico, empleando un diseño experimental en el que se integraron algoritmos estructurados como entrada para sistemas de inteligencia artificial generativa. La población estuvo conformada por fragmentos de código y problemas de programación, utilizando como muestra distintos algoritmos imperativos representativos. La recolección de datos se realizó mediante pruebas de generación automática de código, evaluando criterios de eficiencia, claridad y funcionalidad. Los resultados evidenciaron que el uso de algoritmos imperativos como insumo permitió mejorar la coherencia estructural del código generado, optimizando el proceso de desarrollo y reduciendo el tiempo de producción. Asimismo, se observó una disminución en errores sintácticos y una mayor alineación con estándares de programación profesional. Se concluyó que la incorporación de lógica estructurada en forma de algoritmos redefine la interacción con modelos de inteligencia artificial, favoreciendo la generación de soluciones más precisas y organizadas. Este antecedente se vincula con la presente investigación al sustentar el uso de algoritmos como componente dentro del framework de ingeniería de prompts, contribuyendo a mejorar la calidad del código generado y a reducir la complejidad ciclomática como parte de la mantenibilidad del software.

Gutiérrez Chávez [11], en su investigación de 2024 titulada *Análisis de la aplicación de prácticas ágiles y la gestión de la deuda técnica en el desarrollo de software en MediaLab*, tuvo como objetivo analizar la relación entre las prácticas ágiles y la gestión de la deuda técnica en entornos de desarrollo con plazos ajustados. La investigación fue de tipo aplicada con enfoque cuantitativo y nivel descriptivo, empleando un diseño no experimental. La población estuvo conformada por proyectos de software desarrollados en MediaLab, seleccionándose una muestra de proyectos activos. Para la recolección de datos se utilizaron encuestas dirigidas a desarrolladores y análisis estático mediante la herramienta SonarQube, evaluando métricas relacionadas con la deuda técnica y la calidad del código. Los resultados evidenciaron que la ausencia de estrategias formales para la gestión de deuda técnica incrementa significativamente los costos de mantenimiento y afecta la sostenibilidad del software a largo plazo. Asimismo, se identificó que la presión por cumplir plazos reduce la calidad del código, generando acumulación de deuda técnica. Se concluyó que es necesario implementar enfoques preventivos y herramientas de análisis continuo para mejorar la calidad del software. Este antecedente se vincula con la presente investigación al respaldar el uso de un pipeline de análisis estático como mecanismo para la detección temprana de deuda técnica, contribuyendo al cumplimiento del objetivo específico relacionado con la mejora de la mantenibilidad del software.

2.2 Bases teóricas

2.2.1 Marco Conceptual

2.2.1.1 Ingeniería de Prompts (Prompt Engineering)

Disciplina técnica centrada en el diseño y optimización de instrucciones textuales para guiar modelos de lenguaje hacia resultados específicos. Implica el uso de patrones lógicos que minimizan la ambigüedad sintáctica, permitiendo que la inteligencia artificial genere soluciones alineadas con requerimientos arquitectónicos complejos de software.

2.2.1.2 Mantenibilidad del Software

Atributo de calidad que define la facilidad con la que un sistema puede ser modificado, corregido o mejorado. Según la normativa internacional, comprende subccaracterísticas esenciales como la modularidad y la capacidad de ser probado, asegurando que el código sea sostenible durante su ciclo de vida.

2.2.1.3 Deuda Técnica (Technical Debt)

Costo implícito derivado de priorizar la rapidez de entrega sobre la calidad del diseño estructural. Se manifiesta como la acumulación de soluciones subóptimas que requieren refactorización futura, incrementando la complejidad sistémica y los costos operativos de mantenimiento en entornos de desarrollo profesional de software.

2.2.1.4 Complejidad Ciclomática de McCabe

Métrica cuantitativa utilizada para determinar la complejidad lógica de un programa mediante el recuento de rutas linealmente independientes en su flujo de control. Actúa como un indicador crítico de la mantenibilidad, prediciendo la dificultad de evaluar y modificar fragmentos de código específicos en sistemas informáticos.

2.2.1.5 PromptDebt

Fenómeno emergente en la ingeniería generativa donde la carencia de estándares en las instrucciones deriva en código con alta redundancia y baja cohesión. Representa una degradación estructural que compromete la evolución del software, exigiendo marcos de gobernanza técnica para mitigar su impacto en la infraestructura crítica.

2.2.1.6 Análisis Estático de Código

Técnica de inspección automatizada que evalúa el código fuente sin ejecutarlo para identificar vulnerabilidades y desviaciones de estándares de calidad. Permite la detección temprana de defectos estructurales y la medición precisa de métricas de mantenibilidad mediante herramientas especializa-

das como SonarQube o Radon.

2.2.1.7 Deuda Técnica Autoadmitida (SATD)

Categoría de deuda técnica donde las deficiencias arquitectónicas son reconocidas y documentadas explícitamente por los desarrolladores mediante comentarios o metadatos. En sistemas generados por inteligencia artificial, indica áreas críticas que requieren intervención humana obligatoria para asegurar la portabilidad y eficiencia del producto final.

2.2.2 Bases Teóricas

2.2.2.1 Ingeniería de Software y Calidad del Producto

- **Modelos de calidad de software y evolución:** La ingeniería de software ha evolucionado desde un enfoque centrado en la ejecución hacia la gestión de atributos internos que garantizan la sostenibilidad del producto. Modelos clásicos como los de McCall y Boehm permitieron por primera vez descomponer la çalidad.[en] factores medibles [20]. Esta evolución técnica es la base que permite hoy evaluar si el código generado por inteligencia artificial cumple con los requisitos mínimos de robustez arquitectónica necesarios en entornos de producción industrial.
- **La norma ISO/IEC 25010 y la mantenibilidad:** El estándar ISO/IEC 25010 constituye el eje normativo de este estudio, definiendo la mantenibilidad como la capacidad de un sistema para ser modificado efectiva y eficientemente [2]. Se enfoca en subccaracterísticas clave: modularidad, reusabilidad, analizabilidad, capacidad de ser modificado y capacidad de ser probado. Estas dimensiones permiten cuantificar el impacto de la ingeniería de prompts, determinando si el código resultante facilita o dificulta la evolución del sistema frente a cambios de requisitos.
- **Métricas de calidad interna y complejidad:** Para validar la mantenibilidad, se emplean métricas cuantitativas como la Complejidad Ciclomática de McCabe y el Índice de Mantenibilidad (MI). Estos indicadores evalúan la densidad lógica y el esfuerzo cognitivo requerido para comprender el código [9]. En el desarrollo de microservicios, mantener estos valores dentro de umbrales óptimos es crucial para evitar el colapso del sistema por acumulación de errores arquitectónicos invisibles en la superficie funcional.

2.2.2.2 Inteligencia Artificial Generativa en el Desarrollo de Software

- **Modelos de Lenguaje de Gran Escala (LLMs):** Los modelos fundamentados en arquitecturas de transformadores han revolucionado la programación mediante la predicción probabilística de tokens de código. Estos sistemas, entrenados en vastos repositorios como GitHub, poseen una capacidad emergente para resolver problemas algorítmicos comple-

jos [1]. Sin embargo, su naturaleza estocástica implica que la calidad de la salida es altamente dependiente del contexto proporcionado, lo que genera una variabilidad técnica que debe ser controlada mediante marcos de referencia.

- **Capacidades y limitaciones en la generación de lógica:** Aunque los LLMs demuestran una alta velocidad en la creación de funciones, suelen presentar limitaciones en la visión arquitectónica global y la cohesión de módulos extensos [15]. El riesgo de generar código "verboso.º con redundancia lógica es una constante que afecta la deuda técnica. Esta limitación justifica la necesidad de un framework que no solo genere código, sino que condicione la generación a estándares de calidad prefijados.

2.2.2.3 Paradigma de la Ingeniería de Prompts (Prompt Engineering)

- **Técnicas de estructuración de instrucciones:** La ingeniería de prompts se define como la disciplina de optimizar las entradas de un modelo para obtener salidas precisas. Técnicas como el *Few-Shot Prompting* permiten al modelo aprender de ejemplos previos insertados en la instrucción, mejorando la adherencia a estilos de codificación específicos [14]. Estas técnicas son el insumo principal de la variable independiente en la presente investigación, actuando como el mecanismo de control de calidad.
- **Patrones arquitectónicos de prompts:** El uso de patrones como *Chain-of-Thought* (CoT) obliga al modelo a razonar.ª través de pasos intermedios antes de emitir el código final. Este desglose lógico reduce las alucinaciones técnicas y asegura que la lógica de negocio se traduzca correctamente a código ejecutable [13]. La integración de estos patrones en un framework estructurado permite una generación de microservicios más predecible y alineada con las mejores prácticas de la industria.

2.2.2.4 Deuda Técnica en Entornos Generativos

- **Teoría de la Deuda Técnica y evolución:** La deuda técnica es la metáfora financiera aplicada al software que representa el costo futuro de las decisiones de diseño apresuradas [11]. En la era de la IA, esta deuda se acelera debido a la capacidad de los modelos para producir grandes volúmenes de código subóptimo en segundos. La gestión proactiva de esta deuda es el objetivo central del segundo componente experimental de este estudio.
- **PromptDebt y opacidad estructural:** La "Deuda de Prompt"surge cuando las instrucciones utilizadas para generar el sistema no son sostenibles ni escalables [4]. Esto genera una opacidad en el código fuente

que dificulta su mantenimiento futuro, ya que el desarrollador humano debe lidiar con decisiones de diseño tomadas por un modelo sin documentación clara. El estudio busca mitigar este riesgo mediante la estandarización de la instrucción bajo la norma ISO 25010.

2.2.2.5 Marcos de Trabajo y Evaluación Automatizada

- **Frameworks de gobernanza técnica:** Un framework de ingeniería de prompts actúa como una capa intermedia de gobernanza que garantiza que cada línea de código generada sea evaluada bajo criterios de calidad antes de su integración [17]. Esta estructura permite automatizar la aplicación de reglas arquitectónicas, reduciendo la dependencia del juicio humano individual y asegurando la uniformidad en sistemas críticos de microservicios.
- **Pipelines de análisis estático:** La integración de herramientas de análisis estático en el flujo de desarrollo permite medir en tiempo real el Índice de Mantenibilidad y la Deuda Técnica. El uso de herramientas especializadas permite capturar señales premonitorias de degradación técnica [12]. Esta automatización de la vigilancia de calidad es el componente final del framework propuesto, cerrando el ciclo entre la instrucción (prompt) y la validación normativa.

CAPÍTULO 3
METODOLOGÍA

3.1 Método, tipo y alcance de la investigación

3.1.1 Método de investigación

El método de investigación empleado es el hipotético-deductivo. Este método permite partir de una premisa teórica basada en la ingeniería de software y la norma ISO/IEC 25010, para luego someterla a comprobación mediante la observación empírica de métricas de código. A través de la deducción, se establecen las hipótesis que serán validadas o refutadas mediante el análisis estático de los resultados obtenidos tras la aplicación del Framework de Ingeniería de Prompts propuesto.

3.1.2 Tipo de investigación

La investigación es de tipo aplicada y tecnológica. Es aplicada porque busca resolver un problema concreto de la industria del software: la acumulación de deuda técnica en sistemas generados por IA. Es tecnológica porque el aporte principal consiste en la creación de un Framework (herramienta metodológica) y un pipeline de evaluación automatizado que optimiza el proceso de producción de software, transformando el conocimiento teórico en una solución técnica ejecutable.

3.1.3 Alcance de la investigación

De acuerdo con la Guía Metodológica, el alcance de esta investigación es explicativo. El estudio no se limita a describir cómo la IA genera código o a correlacionar variables, sino que pretende establecer una relación de causalidad. Se busca determinar cómo y por qué la estructuración sistémica de los prompts influye directamente en la reducción de la complejidad ciclomática y en la mejora de los índices de mantenibilidad del producto final.

3.1.4 Diseño de la investigación

El diseño de la investigación es experimental, específicamente bajo un esquema cuasi-experimental con pre-prueba y post-prueba. Se utilizará un grupo de control donde la generación de código se realizará mediante métodos empíricos (prompts abiertos) y un grupo experimental donde se aplicará el Framework de Ingeniería de Prompts diseñado. Las mediciones de pre-prueba y post-prueba se realizarán utilizando herramientas de análisis estático (SonarQube/Radon) para comparar la evolución de la calidad estructural del software en ambos escenarios.

3.1.5 Enfoque de la investigación

La presente investigación adopta un enfoque cuantitativo, debido a que se fundamenta en la medición objetiva de variables mediante métricas de calidad de software, tales como el índice de mantenibilidad, la complejidad ciclomática y la deuda técnica. Los datos obtenidos serán analizados estadísticamente con el fin de contrastar las hipótesis planteadas y determinar relaciones de causalidad entre la variable

independiente y las variables dependientes.

3.2 Materiales y métodos

3.2.1 Materiales

3.2.1.1 Estación de Trabajo de Alto Rendimiento

Se refiere a la unidad de procesamiento centralizada destinada a la ejecución de tareas de cómputo intensivo requeridas para la inferencia de modelos de lenguaje y el análisis estático de código.

Tabla 2. Especificaciones técnicas de la estación de trabajo.

Componente	Especificación Técnica
Arquitectura	x86_64
Procesador	Multinúcleo (Multi-core)
Memoria RAM	Mínimo 16 GB

Función dentro del sistema: Actúa como el host principal para el despliegue de contenedores y la gestión de flujos de datos entre los modelos de IA y las herramientas de métricas.

Uso en la solución: Provee la potencia de cálculo necesaria para procesar las 50 peticiones de generación de código por grupo y ejecutar simultáneamente los servicios de SonarQube.

3.2.1.2 Entorno de Ejecución y Nodos Computacionales

Definido como el ecosistema de software de bajo nivel y red local configurado para el control estricto de variables de procesamiento.

- **Sistema Operativo:** Distribución Linux (específicamente Arch Linux para el clúster de pruebas).
- **Infraestructura:** Nodos computacionales locales configurados en el Laboratorio de Ingeniería de Sistemas de la Universidad Continental, sede Cusco.

Función dentro del sistema: Garantiza un entorno de ejecución aislado que minimiza la latencia de red y asegura la reproducibilidad de los experimentos.

Relación con la arquitectura: Constituye la capa de infraestructura (On-premise) sobre la cual se despliega el clúster de pruebas y los servicios de análisis.

3.2.2 Recursos Lógicos y Modelos de Inteligencia Artificial

3.2.2.1 Modelos de Lenguaje de Gran Escala (LLMs) de Acceso Abierto

Constituyen los motores de inferencia probabilística basados en arquitecturas de transformadores, especializados en la generación de sintaxis y lógica

de programación mediante el procesamiento de lenguaje natural.

Tabla 3. Modelos de lenguaje seleccionados para la investigación.

Modelo	Versión/Parámetros	Aplicación Principal
Llama 3	8B / 70B	Generación de microservicios y lógica compleja.
Mistral	v0.1 / v0.3	Validación de reproducibilidad y consistencia.

Función dentro del sistema: Actúan como el agente generador de código fuente que será sometido a evaluación bajo las métricas de mantenibilidad y deuda técnica.

Uso en la solución: Reciben las instrucciones (prompts) estructuradas bajo los lineamientos del framework basado en la norma ISO/IEC 25010 para producir módulos de microservicios.

Lenguaje y entorno de desarrollo: El experimento se desarrollará utilizando el lenguaje de programación Python, debido a su amplia adopción en entornos de desarrollo asistido por inteligencia artificial y su compatibilidad con herramientas de análisis estático como Radon y SonarQube. Asimismo, se empleará un enfoque basado en microservicios utilizando frameworks ligeros como FastAPI para la generación de módulos evaluables.

3.2.2.2 Entorno de Orquestación y Contenerización

Capa de virtualización a nivel de sistema operativo utilizada para empaquetar y ejecutar los servicios de análisis y modelos de forma aislada y replicable.

- **Motor de contenedores:** Docker para el aislamiento de servicios.
- **Orquestación:** Gestión de dependencias y despliegue de SonarQube y Radon.

Relación con la arquitectura: Permite la integración del pipeline de análisis estático con los nodos computacionales, asegurando que el entorno de experimentación sea independiente del hardware subyacente.

3.2.3 Herramientas de Análisis Estático y Desarrollo

3.2.3.1 Framework de Evaluación de Calidad de Código

Conjunto de herramientas de software destinadas a la inspección automática del código fuente sin necesidad de ejecución, permitiendo identificar vulnerabilidades estructurales.

- **SonarQube (Community Edition):** Plataforma central para la medición de la deuda técnica y densidad de defectos. Para el experimento, se configurará un *Quality Gate* estricto, definiendo como métrica de fallo cualquier módulo generado que supere un ratio de deuda técnica del 5 % o que presente vulnerabilidades críticas en el atributo de mantenibilidad.

- **Librería Radon:** Herramienta de línea de comandos en Python para el cálculo de la Complejidad Ciclomática de McCabe.

3.2.3.2 Ecosistema de Lenguajes de Programación Objetivo

Lenguajes de alto nivel seleccionados para la generación de código, representativos del desarrollo actual de microservicios:

- **Python:** Utilizado para la generación de lógica de procesamiento de datos.
- **TypeScript:** Utilizado para la validación de arquitectura de servicios escalables y tipado fuerte.

3.2.4 Métodos

3.2.4.1 Conceptualización del Framework basado en ISO/IEC 25010

Proceso de traducción de los atributos de calidad definidos en la norma internacional hacia una estructura lógica de instrucciones para modelos de lenguaje. Se centra en la descomposición de la Mantenibilidad en subcaracterísticas como modularidad y analizabilidad.

- **Justificación:** Garantiza que el código generado cumpla con estándares industriales para su evolución futura.
- **Artefactos:** Matriz de trazabilidad Norma-Prompt y Plantilla Maestra del Framework.

3.2.4.2 Aplicación de Patrones Arquitectónicos de Prompts

Implementación de técnicas avanzadas de estructuración para guiar el razonamiento del modelo de IA hacia soluciones óptimas.

- **Chain-of-Thought (CoT):** Descomposición de problemas lógicos en pasos secuenciales.
- **Few-Shot Prompting:** Inclusión de ejemplos de código de alta calidad como referencia arquitectónica.

3.2.5 Desarrollo y Generación de Código de Microservicios

3.2.5.1 Configuración del Entorno de Generación

Preparación de la infraestructura lógica para la ejecución de modelos en un entorno controlado. Incluye el despliegue de contenedores Docker y la calibración de hiperparámetros como la temperatura (0.2) para asegurar determinismo.

3.2.5.2 Ejecución de Pruebas de Generación

Ejecución del protocolo cuasi-experimental dividido en dos fases:

- **Grupo Control (Pre-prueba):** Generación de 50 módulos utilizando prompts empíricos abiertos.
- **Grupo Experimental (Post-prueba):** Generación de módulos aplicando el Framework estructurado.

3.2.6 Pipeline de Análisis Estático y Validación de Calidad

3.2.6.1 Proceso de Evaluación de Métricas de Código

Implementación de un flujo automatizado para la extracción de metadatos técnicos. Se utiliza Radon para la Complejidad Ciclomática y SonarQube para el índice de mantenibilidad y deuda técnica.

3.2.6.2 Validación y Control de Calidad

Procedimiento de verificación del cumplimiento normativo y análisis de significancia estadística para la contrastación de las hipótesis planteadas.

- **Pruebas de Normalidad:** Previo al análisis comparativo, se aplicará la prueba de Shapiro-Wilk (considerando el tamaño de muestra $n = 50$ por grupo) para determinar si los datos métricos presentan una distribución normal.
- **Técnicas de Contraste:** Con base en los resultados de normalidad, se utilizará la prueba paramétrica t-Student (si existe normalidad) o la prueba no paramétrica U de Mann-Whitney (si no existe normalidad) para comparar las medias de los grupos control y experimental.
- **Criterio de Ingeniería:** Asegurar la sostenibilidad tecnológica mediante la reducción del tiempo estimado de remediación.

3.3 Sustento Teórico de la Metodología

La integración de la norma ISO/IEC 25010 dentro de la ingeniería de prompts se fundamenta en la necesidad de establecer métricas de calidad interna en sistemas generados por IA. Según Hou et al.[1], la ausencia de rigor arquitectónico en el uso de LLMs deriva en una deuda técnica acumulada que compromete la mantenibilidad a largo plazo.

A nivel regional, la adopción de estas tecnologías en el sur del Perú requiere marcos de cumplimiento que aseguren la resiliencia organizacional frente a la degradación del software. El uso de herramientas como SonarQube y Radon permite operacionalizar estos estándares internacionales en datos cuantitativos y auditables. El enfoque cuantitativo y el diseño experimental adoptado se sustentan en lo planteado por Hernández-Sampieri et al.[21], quienes establecen que los estudios explicativos requieren la manipulación de variables independientes para observar su efecto en variables dependientes bajo condiciones controladas. Asimismo, el diseño cuasi-experimental permite trabajar con grupos no aleatorizados manteniendo validez interna en contextos aplicados.

CAPÍTULO 4

ASPECTOS ADMISNITRATIVOS

4.1 Presupuesto

El presupuesto del proyecto ha sido estructurado considerando los recursos necesarios para la ejecución del proceso experimental, incluyendo recursos humanos, infraestructura, servicios y costos operativos. Esta distribución permite garantizar la viabilidad técnica y económica del estudio.

Tabla 4. Estructura del presupuesto ajustado del proyecto.

Categoría	**Ítem**	**Cant.**	**Unidad**	**P. Unit (S/.)**	**Total (S/.)**
Recursos Humanos	Investigador principal	6	mes	0	0
	Asistente de investigación	0	mes	0	0
Equipamiento	Estación de trabajo	1	unidad	0	0
	Periféricos	1	lote	200	200
Software	Herramientas open source	1	uso	0	0
	Consumo API LLM	1	uso	0	0
Servicios	Internet	6	mes	100	600
	Electricidad	6	mes	80	480
Otros	Impresión	1	lote	150	150
				TOTAL	**1,430**

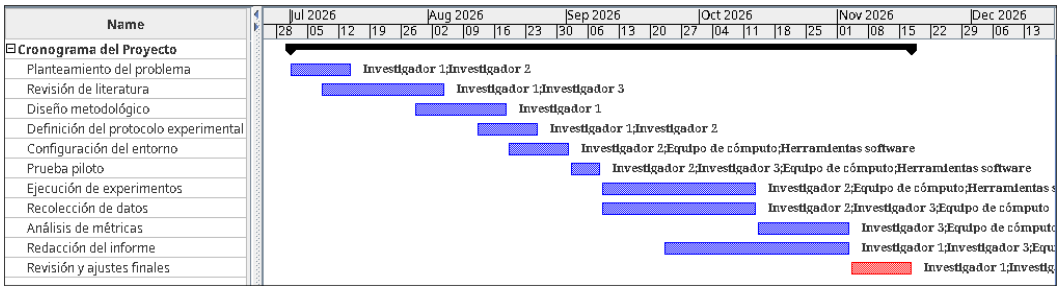
El experimento utilizará modelos de lenguaje de acceso gratuito o con planes académicos, lo cual elimina costos asociados al consumo de API, manteniendo la viabilidad económica del estudio. El proyecto será ejecutado por el investigador como parte de su formación académica, por lo que no se consideran costos salariales.

Criterio de estimación: El presupuesto prioriza el componente de recursos humanos, dado que la investigación se fundamenta en actividades de análisis, diseño experimental y validación. Los costos de software se mantienen reducidos debido al uso de herramientas open source, mientras que se considera un margen de contingencia para cubrir variaciones operativas.

4.2 Cronograma

El cronograma del proyecto ha sido estructurado en un horizonte temporal de 24 semanas, comprendido entre julio y diciembre de 2026. Se ha definido una secuencia lógica que integra actividades de investigación, desarrollo experimental y validación, permitiendo la superposición controlada de tareas para optimizar el tiempo de ejecución.

Figura 1. Cronograma del proyecto elaborado en ProjectLibre



Criterio de planificación: Se ha optado por un enfoque semi-paralelo en la ejecución de actividades críticas, particularmente en la fase experimental, donde la generación de código y la recolección de datos se desarrollan de manera concurrente. Esta estrategia reduce el tiempo total del proyecto sin comprometer la validez de los resultados.

CAPÍTULO 5

RESULTADOS Y DISCUSIÓN

5.1 Presentación de resultado

5.2 Discusión de resultados

CAPÍTULO 6

CONCLUSIONES Y RECOMENDACIONES

ANEXOS

Anexo 1. Matriz de Consistencia

Problemas	Objetivos	Hipótesis	Variables	Metodología
Problema General: ¿Qué efecto tiene la implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 en la mantenibilidad y la deuda técnica del código generado por inteligencia artificial generativa? **Problemas Específicos:** ▪ **PE1:** ¿Qué efecto tiene la aplicación de patrones arquitectónicos de prompts en la complejidad ciclomática de McCabe del código generado? ▪ **PE2:** ¿Qué efecto tiene la implementación de un pipeline de análisis estático en la reducción de la deuda técnica del software generado? ▪ **PE3:** ¿Qué efecto tiene la aplicación del protocolo de validación basado en ISO/IEC 25010 en el índice de mantenibilidad del código generado?	**Objetivo General:** Evaluar el efecto de la implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 en la mantenibilidad y la deuda técnica del código generado por inteligencia artificial generativa. **Objetivos Específicos:** ▪ **OE1:** Evaluar el efecto de la aplicación de patrones arquitectónicos de prompts en la complejidad ciclomática de McCabe. ▪ **OE2:** Evaluar el efecto de la implementación de un pipeline de análisis estático en la reducción de la deuda técnica. ▪ **OE3:** Evaluar el efecto de la aplicación del protocolo de validación basado en ISO/IEC 25010 en el índice de mantenibilidad.	**Hipótesis General:** La implementación de un Framework de Ingeniería de Prompts basado en la norma ISO/IEC 25010 reduce significativamente la deuda técnica y mejora la mantenibilidad del código generado por inteligencia artificial generativa en comparación con métodos empíricos. **Hipótesis Específicas:** ▪ **HIP1:** El código generado mediante el uso de patrones arquitectónicos de prompts presenta menor complejidad ciclomática que el código generado mediante métodos empíricos. ▪ **HIP2:** El código generado con un pipeline de análisis estático presenta menor deuda técnica que el generado sin dicho pipeline. ▪ **HIP3:** El código generado bajo el protocolo de validación basado en ISO/IEC 25010 presenta un mayor índice de mantenibilidad que el código generado sin dicho protocolo.	**Variable Independiente:** ▪ Aplicación del Framework de Ingeniería de Prompts • Nivel 1: Sin framework (método empírico) • Nivel 2: Con framework estructurado **Variables Dependientes:** ▪ Mantenibilidad del Software: • Índice de mantenibilidad • Complejidad ciclomática ▪ Deuda Técnica: • Tiempo de remediación • Densidad de defectos	**Método:** Hipotético-deductivo. **Tipo:** Aplicada / Tecnológica. **Nivel:** Explicativo con enfoque experimental. **Diseño:** Cuasi-experimental (grupo control vs grupo experimental). **Población/Muestra:** Módulos de lógica para microservicios generados por modelos de lenguaje (n = 50 muestras por grupo). **Técnicas/Instrumentos:** Experimentación y análisis estático de código mediante SonarQube y Radon.

REFERENCIAS BIBLIOGRÁFICAS

1. HOU, Xinyi; ZHAO, Yanjie; LIU, Yue; YANG, Zhou; WANG, Kailong; LI, Li; LUO, Xiapu; LO, David; GRUNDY, John y WANG, Haoyu. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* [online]. 2024, volumen 33, número 8, págs. 1-79 [visitado 2026-04-29]. ISSN 1049-331X, ISSN 1557-7392. Disponible desde DOI: 10.1145/3695988.

2. ISO/IEC. *ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Models*. International Organization for Standardization, 2011.

3. LIU, Yue; LE-CONG, Thanh; WIDYASARI, Ratnadira; TANTITHAMTHAVORN, Chakkrit; LI, Li; LE, Xuan-Bach D. y LO, David. Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues. *ACM Transactions on Software Engineering and Methodology* [online]. 2024, volumen 33, número 5, págs. 1-26 [visitado 2026-04-29]. ISSN 1049-331X, ISSN 1557-7392. Disponible desde DOI: 10.1145/3643674.

4. ALJOHANI, Ahmed y DO, Hyunsook. PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects. En: *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering* [online]. Istanbul Turkiye: ACM, 2025, págs. 371-382 [visitado 2026-04-29]. ISBN 979-8-4007-1385-9. Disponible desde DOI: 10.1145/3756681.3756976.

5. INSTITUTO NACIONAL DE ESTADÍSTICA E INFORMÁTICA. *Informe Técnico: Estadísticas de las Tecnologías de Información y Comunicación en las Empresas*. Lima, Perú, 2024. Informe técnico. Instituto Nacional de Estadística e Informática.

6. SGTD. *Propuesta de la Estrategia Nacional de Inteligencia Artificial 2026-2030*. 2024. Informe técnico. Presidencia del Consejo de Ministros. Disponible también desde: https://www.gob.pe/institucion/pcm/informes-publicaciones. Secretaría de Gobierno y Transformación Digital.

7. GOBIERNO REGIONAL DE CUSCO. *Plan de Gobierno Digital 2025-2027*. Cusco, Perú, 2024. Informe técnico. Gerencia Regional de Planeamiento, Presupuesto y Modernización.

8. SIDDIQ, Mohammed Latif; SANTOS, Joanna C. S.; TANVIR, Ridwanul Hasan; ULFAT, Noshin; RIFAT, Fahmid Al y LOPES, Vinicius Carvalho. Using Large Language Models to Generate JUnit Tests: An Empirical Study. En: *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* [online]. 2024, págs. 313-322 [visitado 2026-04-29]. Disponible desde DOI: 10.1145/3661167.3661216. arXiv:2305.00418 [cs].

9. ASFANDYAR, Muhammad Waseem Akhtar. EVALUATING THE IMPACT OF AI CODE GENERATED OF SOFTWARE QUALITY AND OBSERVED STUDY OF MAINTAINABILITY, SECURITY AND PERFORMANCE. *Spectrum of Engineering Sciences* [online]. 2025, volumen 3, número 12, págs. 1379-1389 [visitado 2026-04-29]. ISSN 3007-312X. Disponible desde DOI: 10.5281/ZENODO.18172279.

10. FORMGREN, Björn. *Capabilities of Large Language Models in Generating Complex Code: A Software Architecture Perspective*. Estocolmo, Suecia, 2024. Tesis de maestría. KTH Royal Institute of Technology.

11. GUTIERREZ CHAVEZ, Hector Gonzalo. *Análisis de la aplicación de prácticas ágiles y la gestión de la deuda técnica en el desarrollo de software en MediaLab*. Tacna, Perú, 2025. Trabajo de Investigación de para optar el Grado a Nombre de la Nación de: Maestro en Diseño y Gestión de Proyectos Tecnológicos. Escuela de Posgrado Newman.

12. CUATECONTZI CUAHUTLE, Elizabeth; MEDINA BARRERA, María Guadalupe; CORTÉS MALDONADO, Raúl y BUENO AVENDAÑO, Carlos. Identificación y Medición de Deuda Técnica Autoadmitida en Herramientas de Aprendizaje Profundo: Una Revisión Sistemática. *Innovation and Software* [online]. 2025, volumen 6, número 2, págs. 171-191 [visitado 2026-04-29]. ISSN 2708-0935, ISSN 2708-0927. Disponible desde DOI: `10.48168/innosoft.s24.a287`.

13. MEDIANERO ACOSTA, Jorge. *Algoritmos imperativos como insumo para generar código aplicando técnicas de Prompt Engineering*. Lima, Perú, 2024. Trabajo de Suficiencia Profesional para optar al Título de Ingeniero de Sistemas y Cómputo. Universidad Inca Garcilaso de la Vega.

14. PAZ FERNÁNDEZ, Crishtian Brenon; DIAZ SIFUENTES, Sergio Helí y TORRES VILLANUEVA, Marcelino. Optimización de Modelos de Lenguaje Grande (LLMs) a través del Prompt Engineering. *Innovation and Software* [online]. 2025, volumen 6, número 2, págs. 6-11 [visitado 2026-04-29]. ISSN 2708-0935, ISSN 2708-0927. Disponible desde DOI: `10.48168/innosoft.s24.a212`.

15. AZEEM, Samina; NAVEED, Muhammad Shumail; SAJID, Muhammad y ALI, Imran. AI vs. Human Programmers: Complexity and Performance in Code Generation. *VAWKUM Transactions on Computer Sciences* [online]. 2025, volumen 13, número 1, págs. 201-216 [visitado 2026-04-29]. ISSN 2308-8168, ISSN 2411-6335. Disponible desde DOI: `10.21015/vtcs.v13i1.2043`.

16. ROMERO AGUILAR, Carmela; VARGAS ALARCÓN, Andric Alejandro; BALBOA ZEGARRA, Richar Sandro; ALVARADO SANCHEZ, Victor y PLASENCIA SIFUENTES, Victor Hugo. *Business Consulting: Aplicaciones de marcos de referencia para el desarrollo de software en MS4M S.A.C.* 2024. Tesis de Maestría. Pontificia Universidad Católica del Perú.

17. ORTEGA OVALLE, Maria Teodolinda. Arquitecturas basadas en microservicios: impacto en la mantenibilidad y la evolución de sistemas críticos: Microservices-based architectures: impact on maintainability and the evolution of critical systems. *LATAM Revista Latinoamericana de Ciencias Sociales y Humanidades* [online]. 2026, volumen 7, número 2 [visitado 2026-04-29]. ISSN 2789-3855. Disponible desde DOI: `10.56712/latam.v7i2.5535`.

18. GOBIERNO DEL PERÚ. *Estrategia Nacional de Inteligencia Artificial 2021-2026: Informe de Progreso y Actualización*. Lima, Perú, 2024. Informe técnico. Presidencia del Consejo de Ministros.

19. CORDEIRO, Jonathan; NOEI, Shayan y ZOU, Ying. An Empirical Study on the Code Refactoring Capability of Large Language Models. *ACM Transactions on Software Engineering and Methodology* [online]. 2026, pág. 3801158 [visitado 2026-04-29]. ISSN 1049-331X, ISSN 1557-7392. Disponible desde DOI: 10.1145/3801158.

20. PARDO MESIAS, Samuel Ricardo. *Mantenibilidad de productos de software según el modelo SQUARE ISO/IEC 25000*. Tingo María, Perú, 2018. Tesis de Pregrado. Universidad Nacional Agraria de la Selva.

21. HERNÁNDEZ SAMPIERI, Roberto; FERNÁNDEZ COLLADO, Carlos y BAPTISTA LUCIO, María del Pilar. *Metodología de la Investigación*. 6.ª edición. México D.F.: McGraw-Hill / Interamericana Editores, S.A. de C.V., 2014. ISBN 978-1-4562-2396-0.